MOSIP Biometric Face Liveness & Verification

Universal Team Setup Guide, Cross-Platform Deployment & Technical Reference

1. System Overview & Problem Statement

This system implements the edge face liveness detection and Presentation Attack Detection (PAD) subsystem mandated in **MOSIP Hackathon Problem Statement 04**. It provides secure resident onboarding, operator/supervisor biometric authentication, and multi-layer attack mitigation (printed photos, digital screens, and video replays) using an ISO/IEC 30107 compliant hybrid passive-to-active pipeline. The system runs **100% on-device and offline** with sub-25ms inference latency.

Defense Layer	Engine / Algorithm	Attack Mitigated
Layer 1: Deep Learning PAD	MiniFASNet V2 ONNX (<3ms)	Screen pixel moiré, polarized reflections, paper dot-matrix artifacts
Layer 2: 2D Planar Geometry	Internal Facial Landmark Deformation (std)	Hand-held or shaken printed photos (which behave as rigid 2D planar surfaces)
Layer 3: Dynamic Challenge	Runtime Unpredictable Prompt (Smile / Head Turn)	Pre-recorded video replays on tablets/phones, canned deepfakes
Biometric Matching	SFace 128D Cosine Similarity & YuNet	Real-time identity verification against registered resident database (>=75%)

2. User Interface & Live HUD Overview

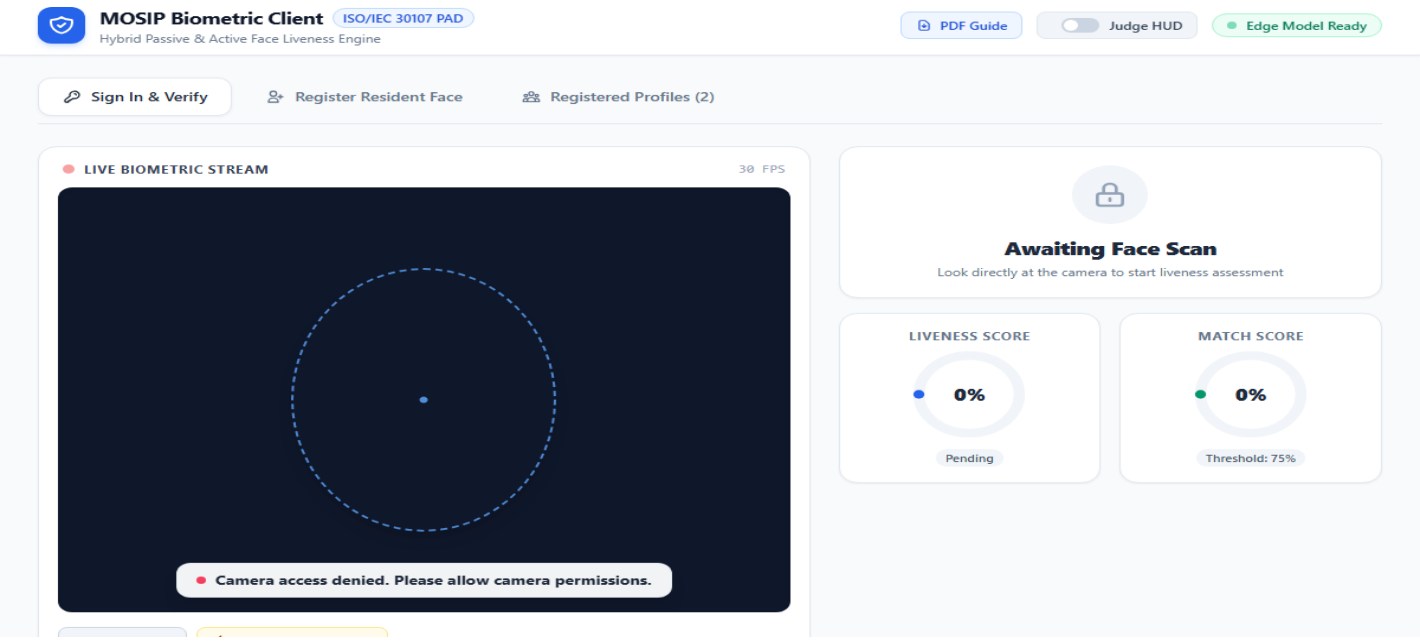


Figure 1: Main 'Sign In & Verify' interface showing live biometric reticle, dual score dials, and system decision status.

3. Universal Team Setup Guide (Windows, macOS, Linux)

Follow these exact steps to run the application on any developer machine. No external cloud credentials or GPUs are required.

Operating System	Command-Line Setup Instructions

Windows (PowerShell / CMD)	`# 1. Navigate to project root`
	`cd "Mosip Hackathon"`
	`# 2. Create & activate virtual environment`
	`python -m venv venv`
	`.\venv\Scripts\Activate.ps1`
	`# 3. Install dependencies from requirements.txt`
	`python -m pip install -r requirements.txt`
	`# 4. Launch the application server`
	`python run_demo.py`

macOS (Apple Silicon / Intel)	`# 1. Navigate to project root`
	`cd "Mosip Hackathon"`
	`# 2. Create & activate virtual environment`
	`python3 -m venv venv && source venv/bin/activate`
	`# 3. Install dependencies`
	`pip install -r requirements.txt`
	`# 4. Launch server`
	`python3 run_demo.py`

Linux (Ubuntu / Debian / Fedora)	`# 1. Install system camera/graphics libs if missing`
	`sudo apt update && sudo apt install -y libgl1-mesa-glx python3-venv`
	`# 2. Create & activate venv`
	`python3 -m venv venv && source venv/bin/activate`
	`# 3. Install dependencies & launch`
	`pip install -r requirements.txt`
	`python3 run_demo.py`

Accessing the Web App: Once launched, open Chrome, Edge, Safari, or Firefox at **http://127.0.0.1:8000** and allow webcam permissions.

4. Resident Face Onboarding Workflow

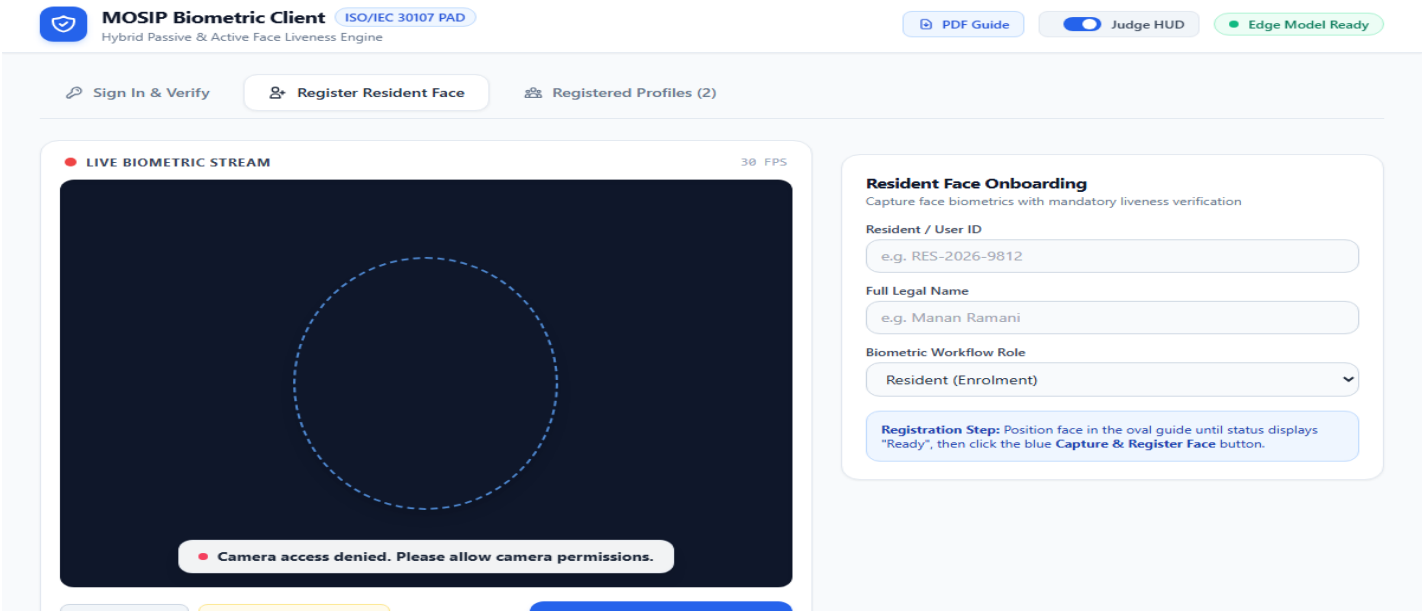


Figure 2: Resident Enrollment tab. Captures full-res frame, executes passive PAD check, and commits 128D embedding to local database.

5. Hybrid Decision Pipeline & PAD Execution Matrix

The system coordinates three autonomous defense checks before granting identity access. Verification is granted strictly when all three stages validate live presence:

Pipeline Stage	Trigger Condition	Target Threshold	Pass Outcome
Stage 1: Passive PAD	Frame ingested from L0/L1 camera	MiniFASNet Live Prob >= 50% & Laplacian in range	Proceeds to identity match & active challenge
Stage 2: 2D Planar Check	Temporal window (15 frames)	Internal deformation std >= 0.0012	Confirms non-rigid 3D biological surface
Stage 3: Active Challenge	Unpredictable runtime selection	Delta MAR >= 0.035 (Smile) or Delta Yaw >= 0.16 (Turn)	Locks in VERIFIED status (Emerald Green)
Stage 4: Identity Match	SFace 128D Cosine Similarity	Cosine score >= 0.363 (Mapped >= 75%)	Access Granted with Resident Name & ID

6. Judge Diagnostic Telemetry HUD

Toggling the **Judge HUD** switch exposes raw telemetry from the multi-layer vision engine in real time:

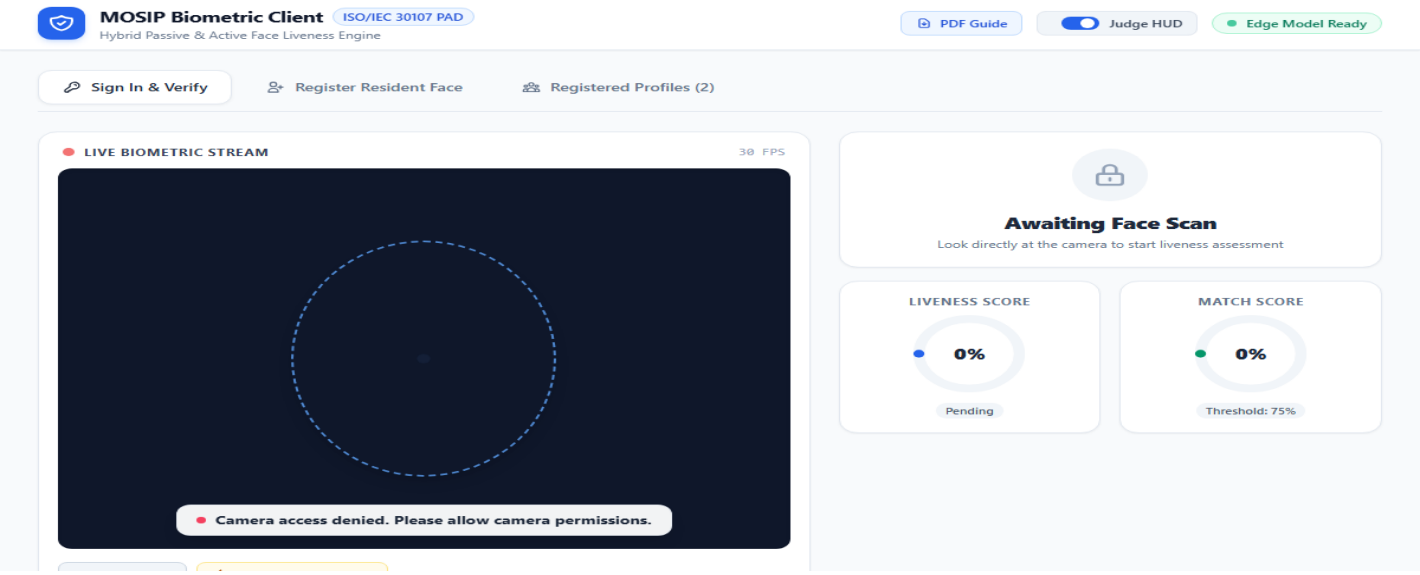


Figure 3: Judge Diagnostic HUD displaying MiniFASNet class, 2D Planar Deformation std, Laplacian variance, and landmark ratios.

7. Registered Biometric Profiles Registry

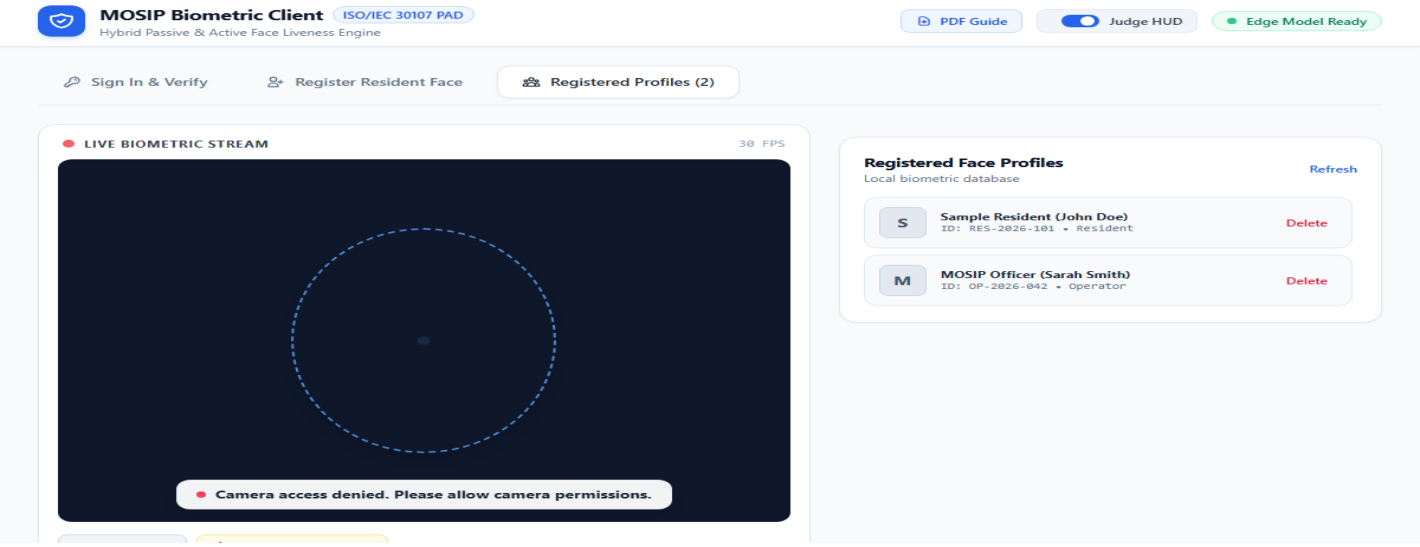


Figure 4: Offline biometric registry storing resident IDs, names, roles, and 128D embeddings.

8. Common Troubleshooting & FAQs for the Team

Issue / Error	Probable Cause	Actionable Resolution
Camera Not Starting	Browser permissions denied or camera locked by Zoom/Teams.	Click the padlock icon in the browser address bar and enable Camera. Close other camera apps, then click 'Restart Camera'.
Port 8000 in Use	A previous Uvicorn background process is still bound to 8000.	Windows: taskkill /F /IM python.exe Mac/Linux: killall python3 Or edit run_demo.py to use port 8001.
Missing Models Error	ONNX model files were not cloned or downloaded.	Verify that models/face_detection_yunet_2023mar.onnx, face_recognition_sface_2021dec.onnx, and minifasnet_pad.onnx exist.
Challenge Timeout	Face was too far from oval reticle or movement was too subtle.	Position face inside the dashed oval and follow prompt (smile naturally or turn head slightly left/right).

9. Project File & Directory Structure

```
Mosip Hackathon/
■■■ models/ # Pre-trained ONNX Edge Models
■ ■■■ face_detection_yunet_2023mar.onnx (YuNet 5-point face detector)
■ ■■■ face_recognition_sface_2021dec.onnx (SFace 128D biometric vector recognizer)
■ ■■■ minifasnet_pad.onnx (MiniFASNet deep learning anti-spoofing)
■■■ app/ # Core Python Backend Services
■ ■■■ face_engine.py (Detection, landmark alignment, 128D cosine matching)
■ ■■■ liveness_engine.py (3-tier Hybrid Passive & Active PAD engine)
■ ■■■ database.py (Local offline JSON template & user registry)
■ ■■■ main.py (FastAPI REST endpoints & WebSocket frame streaming)
■■■ static/ # Light Mode Modern Frontend
■ ■■■ index.html (Biometric HUD layout, reticle oval, and score dials)
■ ■■■ styles.css (Light theme, pulse & attack shake animations)
■ ■■■ app.js (Camera stream, landmark draw, scoring & flow control)
■■■ data/users_db.json (Persistent registered resident database)
■■■ run_demo.py # Single-command launcher (Starts uvicorn on port 8000)
■■■ requirements.txt # Unified Python dependencies
```